

LLM Notes

2026 Spring

# Day 5: VGG 简要分析

deeplearning

主讲: Aaron

Fudan University

## 今日摘要

今天把 VGG 作为一个学习的例子，学习一下其部分细节实现

## 5.1 VGG 代码概览

```
1 class VGG(nn.Module):
2     def __init__(self, features, num_classes=1000, init_weights=
      True):
3         super(VGG, self).__init__()
4         self.features = features
5         self.avgpool = nn.AdaptiveAvgPool2d((7, 7)) # 自适应池化
6         self.classifier = nn.Sequential( # 分类头 (全连接层)
7             nn.Linear(512 * 7 * 7, 4096),
8             nn.ReLU(True),
9             nn.Dropout(),
10            nn.Linear(4096, 4096),
11            nn.ReLU(True),
12            nn.Dropout(),
13            nn.Linear(4096, num_classes),
14        )
15        if init_weights:
16            self._initialize_weights()
17
18    def forward(self, x):
19        x = self.features(x)
20        x = self.avgpool(x)
21        x = torch.flatten(x, 1) # 展平
22        x = self.classifier(x)
23        return x
24
```

```
25     def _initialize_weights(self):
26         for m in self.modules():
27             if isinstance(m, nn.Conv2d):
28                 nn.init.kaiming_normal_(m.weight, mode='fan_out',
29                                         nonlinearity='relu')
30                 if m.bias is not None:
31                     nn.init.constant_(m.bias, 0)
32             elif isinstance(m, nn.BatchNorm2d):
33                 nn.init.constant_(m.weight, 1)
34                 nn.init.constant_(m.bias, 0)
35             elif isinstance(m, nn.Linear):
36                 nn.init.normal_(m.weight, 0, 0.01)
37                 nn.init.constant_(m.bias, 0)
```

Listing 1: VGG 网络主类实现

## 分析

主要值得注意的点在于卷积层的初始化方式，采用了  $\text{fan}_{out}$  方式。前两天提过，在一般的现代神经网络中几乎都采用  $\text{fan}_{in}$  初始化，这里比较特殊。

原因在于 VGG 网络的维度变化是  $3 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$

在方差分析下，卷积层也有近似  $C_{in} \text{Var}(W) \text{Var}(\delta)$  的方差，类似的也有以  $C_{out}$  为系数的梯度方差，这里依旧是类似的权衡，由于没有残差连接，并且输入输出成倍增加，优先解决方差不稳定的问题